五大會計師公會系統整合遷移計畫

目錄

- 一、背景與目標
- 二、現況架構
- 三、目標架構
- 四、風險分析與控制措施
  - 4.1 程式碼層風險
  - 4.2 架構層風險
- 五、目標規格與費用
- 六、實施方法
- 七、人力規劃（AI IDE Agent 協作）
- 八、PRD 切換 SOP
- 九、驗收標準
- 十、待溝通與業務決策事項

一、背景與目標

背景

五個會計師公會（tpecpa、roccpa、khcpa、taiwancpa、tchcpa）目前各自部署於獨立 GCP 專案，架構相同但各自維護，每個組織每月承擔 $560USD$（PRD390 + UAT 170），五個組織合計 $*$ $*$2,800 USD/月**。規模較小的公會基礎設施費用佔比過高，且多套環境分散導致維運複雜、升版需重複執行。

目標

目標	說明
成本降低	整合至單一 GCP 專案，CloudSQL 合併為 2 個共用 instance，預估每月節省 **~66%（約 $1,840 USD）**
維運簡化	統一 CI/CD、統一 LB 與憑證管理、統一監控
品質提升	藉此機會修復現有程式碼中已知的取號競態條件（race condition）bug
手機服務不中斷	Android、iOS、React Native 三個 App 不需重新發版
系統可擴展	正確實作分散式鎖定，未來有需要可安全水平擴展

成功標準

- 所有組織 PRD 環境完成切換，原域名服務正常
- 取號（印鑑、收據、公文、委託）無重複，可跨 instance 安全執行

- 所有排程如期執行、不重複觸發
- 舊 VM 保留兩週後無需回滾，正式下線

二、現況架構

```
[Internet] → DNS → VM（各自獨立 GCP 專案）
                │
                ├─ Nginx :8000 → Tomcat :443（Java Spring Boot WAR）
                ├─ Nginx :9443 → 靜態檔案（Gatsby React build）
                │
                ├─ CloudSQL PostgreSQL（各組織各自 instance，UAT+PRD 共 10 個）
                └─ GCS（各組織各自 bucket，SA JSON key 寫死在程式碼）
```

技術堆疊： Spring Boot 2.4.1 / Java 11 / Gatsby 4 / PostgreSQL / Firebase FCM / SendGrid

手機 App 連線方式（三個 App 共用相同 domain 結構）：

App	連線
Android（5 flavors）	`https://{member | test}.{org}.org.tw:8000/mobile/*`（WebView）
iOS	目前指向 `tpecpa.syscom.tw`，需釐清是否為測試版
React Native	`https://test.tpecpa.org.tw/TACCTG`（含 Tappay callback）

三、目標架構

```
單一 GCP 專案（cpa-consolidated）
│
[Internet]
    │
    ▼
Global HTTPS Load Balancer（共用，1 個 IP）
├─ Port 443：5 org domain × 2 env = 10 host rules → Cloud Run
├─ Port 8000：同上路由（手機 WebView 不需改版）
├─ Certificate Manager：5 份組織自帶憑證（self-managed）
├─ Cloud Armor：1 個 policy，套用全部 org
│
├─ Cloud Run Services（10 個：PRD × 5 + UAT × 5，同一專案）
│   ├─ backend-tpecpa-prd / backend-tpecpa-uat
│   ├─ backend-roccpa-prd / backend-roccpa-uat
│   ├─ backend-khcpa-prd  / backend-khcpa-uat
│   ├─ backend-taiwancpa-prd / backend-taiwancpa-uat
│   └─ backend-tchcpa-prd / backend-tchcpa-uat
│       └─ Cloud SQL Auth Connector（私有 IP）
│
├─ CloudSQL PostgreSQL（共用 instance × 2）
│   ├─ cloudsql-prd（4 vCPU / 15GB / 500GB SSD）
│   │   ├─ DB: tpecpa_prd
│   │   ├─ DB: roccpa_prd
│   │   ├─ DB: khcpa_prd
│   │   ├─ DB: taiwancpa_prd
│   │   └─ DB: tchcpa_prd
│   └─ cloudsql-uat（1 vCPU / 4GB / 100GB SSD）
│       └─ DB: tpecpa_uat / roccpa_uat / khcpa_uat / taiwancpa_uat / tchcpa_uat
│
├─ GCS Buckets（10 個靜態前端，同一專案）
│   ├─ frontend-tpecpa-prd / frontend-tpecpa-uat
│   └─ ...（各組織各 2 個）
│       └─ Cloud CDN（掛在 LB backend bucket）
│
├─ GCS Buckets（檔案儲存，各組織獨立，同一專案）
│   └─ files-tpecpa / files-roccpa / ...（5 個）
│
├─ Secret Manager（集中存放 DB 密碼、GCS SA、SendGrid key）
├─ Artifact Registry（Docker images，5 個 org 共用倉庫）
└─ Workload Identity（取代 GCS SA JSON key 硬編碼）
```

核心技術決定：

決定	選擇	理由
後端容器	Cloud Run（WAR → JAR），10 個 service，同一專案	無需管理 VM，自動重啟，managed SSL termination
前端	GCS + Cloud CDN（10 個 bucket），同一專案	Gatsby 純靜態，無需 Cloud Run，無冷啟動

決定	選擇	理由
流量入口	Global HTTPS Load Balancer（1 個）	統一 IP、統一憑證、port 8000 forwarding、Cloud Armor 整合
CloudSQL	**1 instance PRD + 1 instance UAT，各含 5 個 DB**	大幅降低成本，各 DB 以不同 user 隔離，資料不互通
排程	Cloud Scheduler → HTTP endpoint（移除 @Scheduled）	GCP 原生，無重複執行風險，可觀察性佳
取號安全	@Lock(PESSIMISTIC_WRITE)	DB 層悲觀鎖，跨 instance 也不重複
Session	**JWT + Memorystore Active Session Store**	JWT 處理認證；Redis 管理多裝置 session 白名單 + 即時撤銷
Memorystore	Cloud Memorystore（Redis，JWT session store 用）	Token 有時效特性，Redis TTL 零維護；帳號停用即時生效
憑證	各公會自帶（self-managed）	公會已有既有憑證，GCP 不另收費

四、風險分析與控制措施

4.1 程式碼層風險（現有 VM 就存在）

🔴 CRITICAL：活動收據號重複

- **位置：** `EventApplyService.setReceiptNumber()` （所有組織）
- **問題：** 完全無 sync 機制，Tomcat 多執行緒下已可能重複； `findFirstByReceiptNoNotNullOrderByReceiptNoDesc()` 無 DB 鎖
- **格式：** `N` + 民國年 + 7 位（如 `N1140000001` ），寫入 `EVENT_APPLY` table
- **控制：** 加 `synchronized` 區塊 + `EventApplyRepository` 新增 `@Lock(PESSIMISTIC_WRITE)` 查詢

🔴 CRITICAL：公文編號重複

- **位置：** `DocumentNoService.getNextNo()` 與 `updateMaxNo()` （所有組織）
- **問題：** 無 sync、無 DB 鎖，多執行緒可讀到同一最大值並各自 +1
- **格式：** 年度 + 類型 + 序號，寫入 `DOCUMENT_NO` table，民國 113 年後統一 10 碼
- **控制：** 加 `synchronized` 區塊 + `DocumentNoRepository` 新增 `@Lock(PESSIMISTIC_WRITE)` 查詢

🟡 MEDIUM：印鑑、收據、學分證書號（WaterRepository）

- **位置：** `SequenceNumService` （ `getSealNumber` 、 `getReceiptNumber` 、 `updCreditCertNumber` ）
- **問題：** 有 `synchronized(sealObject / receiptObj / creditObj)` 但無 DB 層鎖；單 JVM 安全，多 instance 下 `sealObject` 各自獨立，無法跨 JVM 互鎖
- **格式：** 民國年 + 3–5 位序號，寫入 `WATER_NUMBER` table；每年元旦 `ResetSeqSchedule` 重置為 0
- **控制：** `WaterRepository` 新增 `@Lock(PESSIMISTIC_WRITE)` 查詢， `SequenceNumService` 改呼叫帶鎖版本

🟡 MEDIUM：會員大會委託編號（agentNumber）

- **位置：** `MemberMeetingController` （現場 QR Code 掃描即時取號）

- **問題：** 有 `synchronized(lockObj)` 但無 DB 鎖；多人同時掃碼在多 instance 下會重複
- **格式：** 3 位流水（001、002…），寫入 `MEMBER_MEETING_CHECK` / `MEMBER_MEETING_AGENT_NUM`
- **控制：** `MemberMeetingCheckRepositoryImpl.findMaxAgentNumber()` 加 `@Lock(PESSIMISTIC_WRITE)`

> **報到編號（實際到場會員）**：已確認為系統外作業，現場工作人員手寫流水號於紙本報到單，系統中 `checkCode` 存會員代碼、 `checkDate` 存時間戳，屬設計如此，無需改動。

4.2 架構層風險（遷移後新增）

🔴 排程架構重構（Cloud Scheduler 方案）

程式碼審查確認共 14–18 個 `@Scheduled` 方法（每個 org），技術團隊決定改採 **Cloud Scheduler → HTTP endpoint** 架構。

排程完整清單（程式碼審查後確認）：

#	Class	Cron	頻率	複雜度	備註
1	BuildMemberFeeSchedule	`10 15 0 5 * ?`	每月5日	HIGH	批次產生費用單
2	BuildMemberFeeSchedule	`10 15 0 1 1 ?`	每年1/1	HIGH	產生年費資料
3	SendMemberFeeNoticeSchedule	DB動態	設定時觸發	MED	寄費用通知信（含背景執行緒）
4	SendMemberFeeNoticeSchedule	DB動態	設定時觸發	MED	未繳費提醒
5	ResetSeqSchedule	`0 0 0 1 1 ?`	每年1/1	LOW	重置流水號
6	MemberStatusSchedule	`10 5 0 * * ?`	每日00:05	HIGH	更新所有會員狀態（全量）
7	MemberStatusScheduleKS	`10 5 0 * * ?`	每日00:05	HIGH	與 #6 同時執行，plugin-based，需合併
8	IFIACLogSchedule	`59 59 23 L 3,6,9,12 *`	每季末	MED	季度財務 log
9	EventNotificationApplySchedule	`0 0 10 * * *`	每日10:00	MED	活動前一日通知
10	EventNotificationApplySchedule	DB動態	設定時觸發	MED	Discord 通知
11	EventMemberScoreSchedule	DB動態	設定時觸發	LOW	Discord 學分通知
12	EventCancelApplySchedule	`0 * * * * *`	**每分鐘**	MED	取消逾期報名（建議改為每5分鐘）
13	DuplicatePaymentCheckSchedule	`0 */30 * * * ?`	每30分鐘	MED	偵測重複付款（含 Tappay API）
14	DocumentNoticeSchedule	`0 0 8 ? * MON-FRI`	工作日8:00	LOW	公文派辦通知
15	CleanPaymentRequestSchedule	`0 2 0 * * ?`	每日00:02	LOW	清除舊付款請求

#	Class	Cron	頻率	複雜度	備註
16	CheckUnpaidMemberFeeSchedule	`0 0 0 * * ?`	每日（台灣省公會）	MED	解除停業
17	PublicSignListSchedule	`0 0 0 1 1 ?`	每年（全聯會）	MED	建立年度公示名單
18	EventHighlightsSchedule	`0 30 0 * * ?`	每日（全聯會/台中）	LOW	清除一年前活動剪影

兩個特殊問題：

- **每分鐘排程（#12）：** 建議改為 5–10 分鐘，邏輯本身冪等安全
- **DB 動態 cron（#3、#4、#10、#11）：** 改用 Cloud Scheduler 後 cron 定義移至 Cloud Scheduler，不再從 DB 讀取，原欄位廢棄

控制方案：Cloud Scheduler + HTTP Internal Endpoint

```
POST /internal/scheduler/{taskName}
Authorization: Cloud Scheduler OIDC Token（僅允許 Cloud Scheduler 呼叫）
```

重構步驟：移除所有 `@Scheduled` → 方法包裝成 `SchedulerController` endpoint → 加 OIDC 驗證 filter → GCP 建立 Cloud Scheduler jobs（5 org × 18 = 90 個 jobs，AI Agent 批次建立）→ **MemberStatusSchedule + KS** 合併為單一 endpoint

🔴 大檔案上傳失敗（Priority 1，上線前必須完成）

- **問題：** Cloud Run HTTP body 上限 **32MB**；現有 Spring 設定允許 100MB。上線後只要會計師上傳稍大的 PDF 或壓縮檔，立即收到 HTTP 413，這不是邊緣案例，是核心業務流程
- **控制：** 改為 **GCS Signed URL** 架構——後端產生帶時效的 Signed URL，前端直接上傳至 GCS，後端只接收 object path metadata；同時根本解決現有 VM CPU/Memory 瞬間爆掉的問題
- **時程：** Week 2 Day 1 以 khcpa（PoC org）實作並驗證，確認 pattern 後 AI Agent 批次套用其餘 4 org；Week 3 UAT 明確列為驗收 gate（>32MB 上傳必測）

🟡 手機 App Port 8000

- **問題：** Android / iOS WebView 連 `:8000`，LB 預設只開 443
- **控制：** LB 額外建立 port 8000 forwarding rule → 同 URL map 路由到各組織 Cloud Run；App 完全不需改版

🔴 Session 狀態 → JWT + Memorystore Active Session Store

程式碼審查確認：目前使用 **opaque token**（BCrypt + timestamp 的 AES hash，存 DB），每個 request `AuthTokenInterceptor` 做一次 DB lookup；HttpSession 存有 **100+ 個呼叫位置、30+ 個 attribute**。

Session attribute 分類與處置決定：

類型	Keys	數量	處置
認證核心	`user` 、 `superAdmin` 、 `isAcctFirmAgent`	3	**移入 JWT claims**
權限控制	`menuFunction` 、 `buttonControl`	2	**AuthTokenInterceptor 從 DB 查詢後存 request attribute**
UI 設定	`keyword` 、 `domainKeyword` 、 `bankName` 、 `otherCpa`	4	**移入 JWT claims**

類型	Keys	數量	處置
搜尋條件快取	19 個 SearchConditionEnum keys	19	**捨棄**（使用者返回列表頁需重新搜尋）
表單狀態	`feeSearchBean` 、 `paymentConfirm` 、 `batchUploadError` 等	9	**捨棄**（多步驟操作中斷後需重填）

控制方案：JWT + Redis Active Session Store（白名單模式）

```
Redis Key pattern：jwt:{userCode}:{jti}   value: "1"   TTL: jwt.expiry（預設 3600s，參數化）
→ 同一帳號多台裝置登入 = 多個不同 jti 的 key 並存，互不干擾

登入流程：
  POST /login → 後端驗證 → 產生 JWT（claims 含 jti = UUID）
  SET jwt:{userCode}:{jti}  "1"  EX {jwt.expiry}
  Web UI   → JWT 存入 HttpOnly Cookie
  Mobile / API → JWT 存入裝置安全儲存，Authorization: Bearer 帶入

每個 Request：
  AuthTokenInterceptor
    1. 驗 JWT 簽名（純密碼學，secret key 存 Secret Manager，無 DB lookup）
    2. EXISTS jwt:{userCode}:{jti}  →  不存在則 401
    3. 存在 → EXPIRE jwt:{userCode}:{jti}  {jwt.expiry}（滑動視窗，重置 TTL）
    4. JWT claims → request.setAttribute("user", LoginInfoBean)
    5. userCode → DB 查詢 menuFunction / buttonControl → request.setAttribute()
  BaseBean.transferSession() → 改從 request.getAttribute() 讀
  → 100+ 個 controller call site 全部不動

登出（單一裝置）：
  DEL jwt:{userCode}:{jti}   ← 只踢這台

帳號停用 / 強制下線（全部裝置）：
  SCAN jwt:{userCode}:*  →  批次 UNLINK
  → 下次任何裝置的 request 即收到 401
```

JWT claims 設計：

```
{
  "sub": "A123456789",
  "jti": "550e8400-e29b-41d4-a716-446655440000",
  "name": "王小明",
  "type": "MEMBER",
  "isAcctFirmAgent": false,
  "org": "taiwancpa",
  "iat": 1714060800,
  "exp": 1714064400
}
```

帳號停用觸發點（三種帳號類型）：

需在以下停用操作完成後，呼叫 `jwtSessionService.revokeAll(userCode)` → SCAN + UNLINK：

- 授權用戶（ `EventAuthUserEntity` ）停用

- 輪值助理停用
- 一般帳號（ `SysComUserEntity` ）停用

> 停用入口的精確位置需掃描各 org 程式碼確認，AI Agent 可批次處理

Token 有效期： 預設 1 小時（ `jwt.expiry=3600` ），參數化可調整；EXPIRE 重置形成滑動視窗，活躍使用者不需重登

Mobile App 相容性： App 收到的 JWT 字串取代原 opaque token，儲存與帶入方式完全不變，**不需 App 重新發版**。維護窗口後舊 opaque token 全部失效，使用者首次使用需重新登入一次（一次性成本）。

額外費用： Cloud Memorystore（Redis）~$40/月；用途為 JWT active session store（比原 Spring Session + Redis 方案的資料量更小，僅存 jti 字串）

🟡 6am 排程 CPU 爆打 CloudSQL

- **問題：** CloudSQL 合併後，5 個 org 若同時在 6am 跑排程，CPU 峰值疊加，可能影響整個共用 instance
- **控制（零成本方案）：** 利用現有 `application_props` table 動態設定，將 5 個 org 的排程時間 **錯開 15 分鐘**，DB 壓力即時除以 5：
  - tpecpa：06:00（維持原設定）
  - roccpa：06:15
  - khcpa：06:30
  - taiwancpa：06:45
  - tchcpa：07:00
- **注意：** 錯開前需確認各 org 排程彼此獨立，無跨 org 資料依賴；確認後僅需更新 DB 值，無需改程式碼
- **後續：** 排程 query 的根本優化（index / 拆批）仍建議在遷移後觀察期內進行，但不作為上線前 gate

🟡 iOS 指向 syscom.tw

- **問題：** iOS `Const.swift` 目前指向 `tpecpa.syscom.tw` （非 org.tw）
- **控制：** 釐清是否為舊版測試 build；若 App Store 版本已是 `member.tpecpa.org.tw` 則不影響

🟡 SSL 憑證人工續約

- **問題：** 各公會自帶憑證，到期需人工重新上傳至 Certificate Manager（GCP 不自動更新）
- **控制：** 建立憑證到期 calendar 提醒（通常 1–2 年）；日後可逐步改為 Google managed cert 自動更新

🟡 CloudSQL 跨專案遷移 + 合併風險

- **問題一：** CloudSQL 目前散在各組織舊 GCP 專案，需搬移至新專案
- **問題二：** 合併為共用 instance 後，一個 org 的 heavy query（尤其 6am 排程）可能影響其他 org 的 DB 效能
- **問題三：** 一個 instance 故障影響全部 5 個 org（現況是各自獨立故障）
- **控制：**
  - 切換時以 `pg_dump / pg_restore` 逐 org 匯入新 instance，sequence 完整保留
  - 各 org 使用獨立 PostgreSQL DB user，權限隔離，無法跨 DB 查詢
  - 6am 排程問題 **必須在降規合併前先修復**（加 index / 拆批），避免影響他人
  - CloudSQL 啟用 High Availability（HA）模式：~+50% 費用，但故障切換自動，RTO < 60 秒
  - 若未來某 org 用量大幅成長，可單獨分出至獨立 instance

五、目標規格與費用

5.1 現況規格（最大組織為基準）

項目	PRD	UAT
Compute Engine	4 vCPU / 16GB / 60GB SSD	2 vCPU / 8GB / 30GB SSD
CloudSQL	4 vCPU / 15GB / 100GB SSD	2 vCPU / 3.75GB / 20GB SSD
CPU 使用率	VM <20%，DB <10%（6am 有短暫 100%）	可忽略
Memory 使用率	VM <50%（~8GB），DB ❤️.2GB	可忽略
每組織月費	**$390**	**$170**
5 組織合計		**$2,800/月**

5.2 目標規格建議

Cloud Run（後端，取代 VM + Tomcat，同一 GCP 專案 10 個 service）：

環境	規格	min	max	說明
PRD（每個 org）	2 vCPU / 2GB	1	3	JVM `-Xmx1536m` ；Cloud Scheduler 確保排程不重複
UAT（每個 org）	1 vCPU / 1GB	**1**	1	見下方說明

⚠️ 必要設定：CPU Always Allocated（兩個環境都需要）

Cloud Run 預設模式下，即使 min=1，只要沒有 HTTP request 進來，CPU 會被立即 throttle 到幾乎為零。排程已改由 Cloud Scheduler 發 HTTP request 觸發，但 JVM 本身（JWT 驗簽、DB connection pool 維持等）仍需要 CPU 正常運作。

必須在每個 service 的 `service.yaml` 加入：

```yaml
spec:
  template:
    metadata:
      annotations:
        run.googleapis.com/cpu-throttling: "false"
```

這個設定讓 CPU 在 instance 存活期間「一律分配」，排程才能正常運作。PRD 和 UAT 都需要設定。

UAT min-instances=1 的原因：

min=0 時若無 instance 在跑，排程無從執行，UAT 測試行為與 PRD 不一致：

- 測試行前通知（活動前 24 小時寄信）→ 排程時間到但無 instance，不觸發
- 測試月費費用單產生 → 同上

設為 min=1 的代價：

- 每個 UAT service：1 vCPU / 1GB always-on ≈ $3/月
- 5 個 UAT services = ~$15/月（已含於費用試算中）

> PDF / Excel 大量產生場景如 OOM，可即時調高至 4GB，不需停機

CloudSQL（合併為 2 個共用 instance，各含 5 個獨立 DB）：

Instance	規格	說明
`cloudsql-prd`	**4 vCPU / 15GB / 500GB SSD**	5 org DB 合住；各 org 各自 DB user，資料完全隔離
`cloudsql-uat`	**1 vCPU / 4GB / 100GB SSD**	5 org UAT DB 合住；用量極低

容量規劃依據：

- PRD Memory：5 orgs × max 3.2GB = 16GB 峰值，但各 org 不會同時峰值；選 15GB 有充分緩衝
- PRD CPU：5 orgs 正常各 <10% of 4 vCPU = 總計 <50%；6am 排程修復後可進一步降規
- PRD Connections：每 org Cloud Run max pool 10 × 5 org = 50 最大連線；遠低於 CloudSQL 上限
- PRD Storage：各 org 100GB → 共用 instance 保守估算 500GB

> 各組織 DB 以不同 PostgreSQL user 連線，DB 層面完全隔離，不共享資料

5.3 費用對比

> **CUD 前提說明：** 整合至單一 GCP 專案後，基礎設施規模確定、用量可預測，具備簽訂 **GCP 3 年 Committed Use Discount（CUD）** 的條件。Cloud Run 3 年 CUD 折扣約 **32%**；CloudSQL 3 年 CUD 折扣約 **52%（僅運算，儲存不適用）**。此為整合計畫才能獲得的額外降本槓桿，5 個獨立專案分散時難以集中採購。CUD 確切折扣率與條件需與 GCP TAM 確認。

項目	現況（5 個獨立 GCP 專案）	遷移後（無 CUD）	遷移後（3 年 CUD）
Cloud Run PRD	5 × ~$115 = **575**	**$325**	**$221** *(−32%)*
Cloud Run UAT	5 × ~$54 = **270**	**$75**	**$51** *(−32%)*
CloudSQL PRD	5 × ~$210 = **1,050**	**$280**	*$185 = (運算 *182 × 48% + 儲存 $98)*
CloudSQL UAT	5 × ~$78 = **390**	**$68**	*$47 = (運算 *40 × 48% + 儲存 $28)*
Load Balancer（port 443 + 8000）	各自計費 ~$30	**$37**	**$37**
Cloud Armor（WAF + Rate Limit）	**$0**	**$45**	**$45**
GCS 前端 + CDN（10 bucket）	含在 VM 內	**$80**	**$80**
GCS 檔案儲存（5 org bucket）	各自計費	**$20**	**$20**
Cloud Memorystore（Redis，JWT active session store）	**$0**	**$40**	**$40**
Artifact Registry / Secret Manager	**$0**	**$15**	**$15**
其他（egress、static IP）	~$515	**$75**	**$75**
月費總計	**$2,800**	**~$1,060**	**~$816**

項目	現況（5 個獨立 GCP 專案）	遷移後（無 CUD）	遷移後（3 年 CUD）
年費總計	**$33,600**	**~$12,720**	**~$9,792**
3 年總計	**$100,800**	**~$38,160**	**~$29,376**

節省摘要：

比較基準	每月節省	每年節省	3 年節省
現況 → 遷移（無 CUD）	**~$1,740（62%）**	~$20,880	~$62,640
現況 → 遷移 + 3 年 CUD	**~$1,984（71%）**	~$23,808	**~$71,424**
無 CUD → 有 CUD（純 CUD 效益）	~$244（23%）	~$2,928	**~$8,784**

> CloudSQL 由 10 個 instance → 2 個 instance 是最大節省來源（$1,440 \rightarrow$348）。3 年 CUD 在現有整合規模下是近乎無風險的承諾：原有 5 個組織的基礎設施已驗證此用量，合約簽 3 年正好對齊 CUD 週期。

六、實施方法

6.1 程式碼修復（5 組織，AI Agent 並行）

CRITICAL 修復（需先於 Docker 遷移在現有環境驗證）：

修復點	異動位置	作法
活動收據號	`EventApplyRepository` 、 `EventApplyService`	新增 `@Lock(PESSIMISTIC_WRITE)` 查詢； `setReceiptNumber()` 加 `synchronized`
公文編號	`DocumentNoRepository` 、 `DocumentNoService`	同上； `getNextNo()` 與 `updateMaxNo()` 加 `synchronized`

MEDIUM 修復（容器化時一併完成）：

修復點	異動位置	作法
印鑑 / 收據 / 學分號	`WaterRepository` 、 `SequenceNumService`	新增帶鎖查詢；Service 改呼叫
委託編號	`MemberMeetingCheckRepositoryImpl`	`findMaxAgentNumber()` 加 `@Lock(PESSIMISTIC_WRITE)`
排程架構重構	見下方 6.2a	Cloud Scheduler 方案
JWT 無狀態認證重構	見下方 6.2b	移除 HttpSession，真正無狀態

6.2a 排程架構重構（Cloud Scheduler）

每個 `@Scheduled` 方法重構為 HTTP endpoint：

```java
// 原本（移除）
@Scheduled(cron = "10 15 0 5 * ?")
public void buildMemberFee() { memberFeeService.generatePaymentData(); }

// 新增 SchedulerController
@RestController
@RequestMapping("/internal/scheduler")
public class SchedulerController {

    @PostMapping("/buildMemberFee")
    public ResponseEntity<String> buildMemberFee(HttpServletRequest req) {
        verifyCloudSchedulerToken(req);   // OIDC token 驗證
        memberFeeService.generatePaymentData();
        return ResponseEntity.ok("done");
    }
}
```

OIDC 驗證 filter（確保只有 Cloud Scheduler 可呼叫）：

```java
// 驗證 request header Authorization: Bearer <Google OIDC token>
// 確認 audience = Cloud Run service URL
```

Cloud Scheduler jobs（AI Agent 批次建立，每 org 一套）：

```yaml
# scheduler-khcpa.yaml（AI 生成後 gcloud 批次建立）
- name: buildMemberFee-khcpa
  schedule: "10 15 0 5 * ?"
  timeZone: Asia/Taipei
  httpTarget:
    uri: https://backend-khcpa-prd-xxx.run.app/internal/scheduler/buildMemberFee
    httpMethod: POST
    oidcToken:
      serviceAccountEmail: scheduler-sa@cpa-consolidated.iam.gserviceaccount.com
```

特別處理：DB 動態 cron（#3、#4、#10、#11）
這 4 個排程原本從 `application_props` table 讀取 cron；改為 Cloud Scheduler 後，cron 寫在 Cloud Scheduler job definition。現有 DB 設定欄位保留（不刪除），但 app 不再讀取。若需調整執行時間，直接在 Cloud Scheduler Console 修改。

6.2b JWT 無狀態認證重構

目標： 以 JWT 完全取代 opaque token + HttpSession，達成真正的無狀態架構，不需 Memorystore。

pom.xml 新增（移除 spring-session-data-redis）：

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

Secret Key 管理： JWT signing key 存入 Secret Manager（`jwt-secret-{org}-prd`），Cloud Run 啟動時讀入環境變數 `JWT_SECRET`。

Token 有效期設定（參數化）：

```
# application.properties
jwt.expiry=3600   # 秒，預設 1 小時，可調整
spring.redis.host=${REDIS_HOST}
```

新增 JwtSessionService（核心元件）：

```java
@Service
public class JwtSessionService {
    private static final String KEY_PREFIX = "jwt:";

    // 登入時呼叫
    public void register(String userCode, String jti, long expirySec) {
        redisTemplate.opsForValue()
            .set(KEY_PREFIX + userCode + ":" + jti, "1",
                 Duration.ofSeconds(expirySec));
    }

    // 每個 request 呼叫（驗存在 + 滑動視窗）
    public boolean isActive(String userCode, String jti) {
        String key = KEY_PREFIX + userCode + ":" + jti;
        Boolean exists = redisTemplate.hasKey(key);
        if (Boolean.TRUE.equals(exists)) {
            redisTemplate.expire(key, Duration.ofSeconds(expiry)); // 滑動視窗
            return true;
        }
        return false;
    }

    // 帳號停用 / 強制下線（踢所有裝置）
    public void revokeAll(String userCode) {
        Set<String> keys = redisTemplate.keys(KEY_PREFIX + userCode + ":*");
        if (!CollectionUtils.isEmpty(keys)) {
            redisTemplate.unlink(keys); // 非阻塞刪除
        }
    }

    // 登出（單一裝置）
    public void revoke(String userCode, String jti) {
        redisTemplate.unlink(KEY_PREFIX + userCode + ":" + jti);
    }
}
```

異動的 5 個類別（其餘 100+ 個 call site 不動）：

類別	異動內容
`UserService.GenAccessToken()`	改為產生 JWT（含 jti）；登入成功後呼叫 `jwtSessionService.register()`
`AuthTokenInterceptor.preHandle()`	① 驗 JWT 簽名；② `jwtSessionService.isActive()` 確認 Redis 白名單；③ claims → `request.setAttribute()` ；④ userCode → DB 查詢 menuFunction/buttonControl
`BaseBean.transferSession()`	改從 `request.getAttribute("user")` 讀取，移除所有 `session.getAttribute()` 呼叫
`LoginController`	登入成功後以 `Set-Cookie: access_token=<jwt>; HttpOnly; Secure; SameSite=Strict` 回應（Web UI 路徑）
三種帳號停用入口	在授權用戶 / 輪值助理 / 一般帳號的停用邏輯中，加入 `jwtSessionService.revokeAll(userCode)`

pom.xml 新增（Redis client）：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

AI Agent 批次清理（非必要但推薦）： 移除所有 controller 中對 28 個 UI 狀態 bean 的 `session.setAttribute()` / `session.getAttribute()` 呼叫。

6.2 容器化（後端）+ GCS Signed URL 重構

Week 2 Day 1：以 khcpa 同步實作以下所有項目，確認 pattern 後套用其餘 4 org

每個組織後端變更：

- `pom.xml` ：packaging 改 `jar` ，移除 Tomcat SSL connector 設定，移除 100MB multipart 限制
- `Dockerfile` ：多階段 build（Maven → `openjdk:11-slim` ，JVM flags 設定）
- `application-prd.properties` / `application-uat.properties` ：DB URL、GCS SA 改讀環境變數
- `service.yaml` ：Cloud Run 規格、Cloud SQL connector、Secret Manager 掛載（含 `jwt-secret` ）、 **`cpu-throttling: "false"`**

GCS Signed URL 重構（Priority 1，與容器化同步完成）：

- **後端：** 新增 `/api/upload/signed-url` endpoint，接受檔案名稱與 Content-Type，回傳帶時效的 GCS Signed URL
- **後端：** 原接收 multipart file 的 controller 改為接收 GCS object path，從 GCS 讀取後續處理
- **前端：** 所有有上傳功能的頁面改為兩段式：① 打 API 拿 Signed URL → ② 前端直傳 GCS

> 需事先確認 5 個 org 的上傳入口是否走同一套 controller，若是則改一次批次套用

6.3 前端靜態化

Gatsby build output 部署至 GCS bucket：

- 每個組織 UAT + PRD 各一個 bucket，10 個共
- `const.js` API URL 確認（domain 不變則不需改）
- GitHub Actions 改為 `gatsby build` → `gsutil rsync`

6.4 Load Balancer 設定

- 1 個 Global LB，2 個 forwarding rule（port 443 + port 8000）
- 10 個 host rules（5 org × 2 env），各自路由至對應 Cloud Run service
- Certificate Manager 上傳 5 份各公會自帶憑證（self-managed）
- 後端靜態：10 個 GCS backend bucket

6.5 Cloud Armor

- 1 個 security policy 掛在 LB 上，套用所有組織
- 規則：Rate limiting × 2、IP 封鎖、Geo 限制、OWASP WAF（SQL injection / XSS）× 6

七、人力規劃（AI IDE Agent 協作）

採用 AI IDE Agent（Claude Code）大量協作，由 AI 負責程式碼生成與跨組織批次套用，人員負責決策、review 與驗證。

時程概覽（4 週開發 + 1 個切換週末）

```
Week 1：程式碼修復 + 第一組織 PoC（khcpa）
Week 2：批次容器化（5 org）+ 前端靜態化 + LB 設定
Week 3：UAT 切換驗收（5 org 同步）
Week 4：PRD 切換準備 + 週六凌晨單一窗口同時上線
```

各 Phase 工時估算

Phase	工作內容	AI 負責	人員負責	預估時間
1	基礎設施建立（GCP 專案、IAM、Secret Manager、LB 骨架、Cloud Armor）	生成 gcloud / Terraform 腳本	執行、review、權限確認	**0.5 天**
2	CRITICAL 修復（EventApply、DocumentNo，先在現有環境驗證）	生成修復程式碼，5 org 批次套用	Review、測試取號不重複	**0.5 天**
3a	排程架構重構（Cloud Scheduler）：khcpa PoC — 移除 @Scheduled、建立 SchedulerController（18 endpoints）、OIDC 驗證、Cloud Scheduler jobs YAML	生成所有 endpoint 程式碼、jobs YAML	Review、部署、觸發測試每個排程	**1 天**
3b	排程重構批次套用（其餘 4 org）	AI 套用 PoC 模式，生成差異設定	Review、確認差異、各 org jobs 建立	**0.5 天**
3c	JWT + Redis Active Session 重構：khcpa PoC — jjwt 依賴、spring-data-redis、JwtSessionService（register/isActive/revokeAll）、UserService（JWT 產生）、AuthTokenInterceptor（驗簽 + Redis whitelist）、BaseBean（session → request attr）、LoginController（HttpOnly Cookie）、三種帳號停用入口加 revokeAll、批次清除 28 個 UI 狀態 session call	生成 JwtSessionService + 5 個核心類別改寫、AI 批次掃描帳號停用入口與 session UI 狀態呼叫	Review、JWT 登入測試（Web/API/Mobile 三路徑）、帳號停用立即失效測試、Memorystore 連線測試	**1.5 天**
3d	JWT 重構批次套用（其餘 4 org）	AI 套用 PoC 模式，生成差異設定（org 名稱、secret key 名稱）	Review、確認 Secret Manager key 名稱正確	**0.5 天**
4	第一組織 PoC 容器化（khcpa）：WAR→JAR、Dockerfile、env profile、	生成所有程式碼與設定檔	Review、Cloud Run 部署、功能驗證、>32MB 上傳測試	**1 天**

Phase	工作內容	AI 負責	人員負責	預估時間
	@Lock、cpu-throttling、GCS Signed URL（後端+前端）、service.yaml			
5	其餘 4 組織批次容器化（AI 套用 PoC 模式）	自動生成差異設定	Review、確認差異、部署	**0.5 天**
6	前端靜態化（10 org × 2 env bucket、CI/CD 更新）	生成 bucket 設定、workflow	Review、執行	**0.5 天**
7	LB 完整設定（URL map、SSL 憑證、port 8000、health check）	生成 URL map 設定	協調各公會提供憑證、DNS 驗證、測試	**0.5 天**
8	UAT 切換（5 org：DB export/import、GCS rsync、DNS 切換）	生成切換腳本	執行、驗收測試	**2 天**
9	PRD 切換（5 org 單一維護窗口，詳見 SOP）	監控與日誌分析輔助	執行 SOP、驗收	**1 個週末（5h）**

開發工期合計：約 9 個工作天
切換工期：1 個週末（週六凌晨 01:00–06:00，5 小時）
所需人力：1 位工程師（操作 AI Agent）+ 1 位指揮兼 QA，共 2 人
總曆法時間：約 5 週

工作範圍	傳統人力估算	AI 協作估算	節省
基礎遷移（Phase 1–2, 4–8）	~19 天（152h）	5.5 天（44h）	~71%
排程重構（Cloud Scheduler，5 org × 18 endpoints = 90 jobs，Phase 3a–3b）	~10 天（80h）	1.5 天（12h）	~85%
JWT + Redis Active Session 重構（Phase 3c–3d）	~12 天（96h）	2 天（16h）	~83%
合計	**~41 天（328h）**	**9 天（72h）**	**~78%**

> 傳統估算以單人有效工時計算（不含學習成本）；AI 協作由 AI 負責程式碼生成與批次套用，人員負責 review 與驗證，整體壓縮比約 4.5x。

八、PRD 切換 SOP

切換前 1 週

- ☐ 新專案 `cloudsql-prd` instance 建立完成，各 org DB 與 user 預先建立
- ☐ Cloud Run service 部署完成，用 `.run.app` URL 冒煙測試通過（含取號測試）
- ☐ GCS bucket 建立，完成第一次大量預同步
- ☐ LB URL map 設定完成，DNS 尚未切換
- ☐ **客戶溝通：** 確認 SSL 憑證方案（客戶自備 vs. GCP 代管）
- ☐ **客戶溝通：** 確認 SendGrid 訂閱歸屬與維運窗口
- ☐ DNS TTL 降至 60 秒（需 48 小時前完成，讓舊 TTL 失效）

- [ ] 維護通知寄送各公會

切換前 24 小時

```
# GCS 預同步（不停機，先搬大部分檔案）
gsutil -m rsync -r gs://old-org-bucket gs://new-org-bucket
```

切換策略：五組織單一維護窗口，同時停機同時上線

五個組織同進退，採**單次維護窗口**，所有 org 並行處理。由 AI Agent 同時操作 5 個 org 的指令，工程師負責監控與決策，指揮兼任後續 QA。

所需人力： 1 位工程師（操作 AI Agent 並行執行）+ 1 位指揮兼 QA
維護窗口： 週六凌晨 01:00–06:00（5 小時，含緩衝）

切換當日流程（5 組織並行，統一指令）

```
T+00:00  【全體同步】5 個 VM 同時停止 Tomcat
         [各 org 負責人] sudo systemctl stop tomcat9
         → 確認 5 個 org 均已停止後，指揮下令進入下一階段

T+00:10  【全體並行】AI Agent 同時發出 5 個 CloudSQL Export 指令
         gcloud sql export sql {tpecpa|roccpa|khcpa|taiwancpa|tchcpa}-prd-instance \
           gs://migration-temp/{org}-prd-$(date +%Y%m%d).sql.gz \
           --database={org}_prd --offload
         → AI Agent 5 個 terminal 並行，工程師監控進度

T+00:30  確認 5 個 export 完成（AI Agent 檢查各檔案大小，回報異常）

T+00:35  【全體並行】AI Agent 同時發出 5 個 DB Import 指令
         gcloud sql import sql cloudsql-prd \
           gs://migration-temp/{org}-prd-*.sql.gz \
           --database={org}_prd
         → CloudSQL 支援同時匯入不同資料庫；工程師觀察進度

T+01:15  【全體同步】AI Agent 連線 5 個 DB，平行執行驗證腳本
         SELECT sequencename, last_value FROM pg_sequences ORDER BY sequencename;
         SELECT relname, n_live_tup FROM pg_stat_user_tables ORDER BY n_live_tup DESC;
         → AI 彙整結果回報工程師，指揮確認全數 OK 才進入下一步

T+01:30  【全體並行】AI Agent 同時發出 5 個 GCS rsync 指令
         gsutil -m rsync -r -d gs://old-{org}-files gs://new-{org}-files

T+01:45  【全體並行】AI Agent 同時更新 5 個 Cloud Run 環境變數並重新部署

T+02:00  【全體並行】AI Agent + 指揮兼 QA 同時對 5 個 org 冒煙測試
         - 登入 API
         - 取號測試（印鑑、收據）
         - GCS 檔案讀取
         → 全數 OK 才切 DNS

T+02:15  【全體同步】5 個 org DNS 同時切換
         → 所有 member.{org}.org.tw A record 同時改指向新 GLB IP
         → TTL=60s，1 分鐘內全面生效

T+02:20  驗證 DNS 生效（AI Agent 對 5 個 domain 同時執行 nslookup / curl）

T+02:30  指揮兼 QA 對 5 個 org 執行全功能驗收
         - 瀏覽器登入
         - Port 8000 WebView（手機 App 路徑）
         - 上傳檔案（GCS Signed URL）

T+03:00  全數正常，宣告切換成功
         各 VM 保留停止狀態（2 週後統一刪除）
```

PRD 切換時程（單一週末）

時段	說明
Week 4（開發完成週）	UAT 所有 5 org 切換驗收完成
第 5 週六 01:00	**PRD 單一維護窗口，5 org 同時切換**
第 5 週六 06:00	預計完成，保留 1 小時緩衝

> 若其中一個 org 發生問題，不影響其他 org 的判斷；單一 org 可獨立 rollback，其餘繼續服務

Rollback 程序（10 分鐘內完成）

```
1. DNS 改回舊 VM IP（TTL=60s，1 分鐘生效）
2. 舊 VM 重啟 Tomcat：sudo systemctl start tomcat9
3. 確認 https://member.{org}.org.tw 回到舊服務
```

舊 VM 切換後保留 **2 週**，確保 rollback 窗口充足後再刪除。

九、驗收標準

項目	驗收方式
取號不重複	模擬 50 concurrent 請求同時申請印鑑，確認無重複號碼
排程不重複執行	觀察 Cloud Scheduler 執行紀錄，確認同一 job 只觸發一次；GCP Console → Cloud Scheduler → Job 執行歷程
手機 App 正常	Android / iOS / RN 各登入、WebView 載入、port 8000 路徑通暢
大檔案上傳	測試 >32MB 檔案走 GCS Signed URL 成功上傳
排程如期執行	在 UAT 觀察至少 1 個完整排程週期
DNS 與憑證	所有 10 個 domain 均正常 HTTPS，憑證有效期確認
壓力測試	k6 模擬 50 並發，確認 Cloud Run auto-scaling 在 max=3 範圍內正常
帳號停用即時失效	停用授權用戶 / 輪值助理 / 一般帳號後，持有舊 JWT 的 request 立即收到 401；確認 Redis key 已被 UNLINK
費用驗證	首月帳單確認落在 $1,000$–1,150 區間（含 Memorystore）
業務歸屬確認	確認憑證與 SendGrid 責任歸屬

十、待溝通與業務決策事項

為確保遷移後維運權責清晰，需於切換前與五個公會窗口確認以下決策：

10.1 SSL 憑證管理方案

方案	優點	缺點	建議
方案 A：GCP 代管憑證	**免費**、**全自動續約**、無須人工上傳、零斷線風險。	僅支援域名驗證 (DV)，不支援綠色網址列 (OV/EV)。	**推薦首選**。省錢且無事故風險。
方案 B：客戶自備憑證	客戶可繼續使用既有購買之商用憑證。	**需人工維護**。到期前須手動上傳，忘記續約會導致網站掛掉。	若客戶對商用標章有堅持再採用。

- **需確認點**：若改用方案 A，客戶需同意在 DNS 切換後，由 GCP 自動簽發憑證。

10.2 SendGrid 郵件服務歸屬

歸屬方	費用支付	維運查信 (Log) 責任	說明
客戶自行續約	客戶直接對廠商支付。	客戶/維管窗口自行進後台查。	我們僅負責串接 API，不介入寄信內容追蹤。
由我們負責代管	費用由我們代繳（含入合約）。	我們協助查詢寄信狀態。	**需耗費顧問點數或維護合約**。若信件沒寄到，由我們負責追查原因。

- **需確認點**：確認各公會目前的方案，若需我們代管，應同步調整維護合約內容。

十一、附件：跨組織進階功能選項（整合後加值選項）

> 本附件所描述之功能**不包含於基礎整合計畫工時內**，屬整合完成後可另行洽談的加值選項。整合後的單一 GCP 專案架構（共用 Memorystore Redis、統一 Cloud Run、同一 CloudSQL instance）為以下功能提供了實作基礎，若未整合則無法以此方式實現。

11.1 跨組織密碼同步

功能說明

若同一位用戶（以身分證字號 `user_code` 為唯一識別）同時擁有多個公會的有效帳號，在任一公會修改密碼後，系統自動將新密碼同步至其所有有效帳號的公會。

技術可行性依據（整合後方可實現）

- 整合後 5 個 Cloud Run services 均在同一 GCP 專案，service-to-service OIDC 認證已作為 Cloud Scheduler 基礎建設一併建立，本功能可直接複用
- `user_code` （身分證字號）在各公會均為主鍵，跨組織識別無歧義
- 各公會 `SYS_COM_USER.ENABLED` 欄位可精確篩選有效帳號，避免同步至已停用帳號

推薦方案：應用層非同步內部 API 同步

```
使用者在 A 公會修改密碼（現有邏輯不變）
    ↓ 非同步（不影響使用者等待時間）
CrossOrgSyncService 並行呼叫其餘 4 個 org Cloud Run：
    POST /internal/crossorg/sync-password
    Authorization: Cloud Run OIDC Token
    Body: { "userCode": "A123456789", "passwordHash": "BCrypt hash" }
        ↓ 各 org：若 user_code 存在且 enabled=true → 更新密碼，否則跳過
```

評估面向	說明
費用影響	**$0**（同一 GCP 專案內 Cloud Run 互呼，egress 費用可忽略）
新增基礎設施	無
一致性	即時同步（非同步後台執行，約 1 秒內完成）；服務中斷時 retry 3 次，失敗記 Cloud Logging
安全性	OIDC token 驗證，僅允許同專案內的 service account 呼叫

備選方案：Cloud Pub/Sub 事件驅動（保證送達、真正解耦，+~$5/月，適合對一致性要求更高的場景）

工時估算

工作項目	傳統工時	AI 協作工時
CrossOrgSyncController + Service（khcpa PoC）	2 天	0.5 天
UserRepository 新增帶條件更新查詢（5 org）	1 天	0.25 天
AI 批次套用其餘 4 org	1–2 天	0 天（AI 執行）
整合測試（改密碼 → 驗另一 org 可用新密碼）	1 天	0.25 天
合計	**5–6 天**	**~1 天**

11.2 跨組織單一登入（SSO）

功能說明

使用者登入任一公會後，點擊前往其他公會的連結時：

- **有帳號**的公會：直接進入，無需重新輸入帳密
- **無帳號**的公會：跳至該公會的登入頁面

僅限 Web 端（桌機瀏覽器），不影響 App WebView 及 React Native 路徑。

技術可行性依據（整合後方可實現）

整合計畫已規劃 **Memorystore Redis** 作為 JWT active session store（`jwt:{userCode}:{jti}` 白名單），5 個 org 共用同一 Redis instance。跨 org SSO 僅需在此 Redis 上新增一個 `xorg:` 前綴的一次性 token key pattern，**零額外費用**。

核心挑戰說明

JWT 存於 HttpOnly Cookie，Cookie 具有 domain 限制： `tpecpa.org.tw` 的 Cookie 無法自動帶到 `khcpa.org.tw` 。即使五個 org 共用同一 JWT signing key，此問題仍然存在。解法是透過後端產生短效一次性 token，經由 redirect 完成跨 domain 的 session 建立。

推薦方案：Redis 一次性跨 org Token Exchange

新增 Redis key pattern（疊加於現有 `jwt:` 之上，不衝突）：

```
xorg:{UUID}  →  { "userCode": "A123456789", "returnPath": "/dashboard" }  TTL: 30 秒
```

完整流程（以從台北公會前往高雄公會為例）：

```
① 使用者點擊「前往高雄公會」連結
   → 瀏覽器 GET https://member.tpecpa.org.tw/crossorg/redirect?target=khcpa&path=/dashboard

② [tpecpa backend]
   - 驗 JWT（從 HttpOnly Cookie 讀取）+ 確認 Redis 白名單有效
   - 生成 token = UUID（122 bits 亂數，不可猜測）
   - Redis SET xorg:{token} {"userCode":"A123456789","returnPath":"/dashboard"} EX 30
   - HTTP 302 → https://member.khcpa.org.tw/auth/crossorg?token={token}

③ [khcpa backend]
   - Redis GET xorg:{token}  →  取得 userCode
   - Redis DEL xorg:{token}   （一次性消費，防重放）
   - SELECT WHERE user_code=? AND enabled=true
   - 有帳號 → 產生 khcpa JWT、SET Redis 白名單、Set-Cookie → 302 到 /dashboard
   - 無帳號 → 302 → /loginPage
```

前端改動範圍： 僅需將「前往其他公會」的連結目標改為 `/crossorg/redirect?target=xxx&path=yyy` ，其餘頁面邏輯不動。

評估面向	說明
費用影響	**$0**（Redis 已規劃；xorg key 極小，不影響 tier）
新增基礎設施	無
使用者體驗	一次 redirect（~100ms），幾乎無感
安全性	UUID 一次性、30 秒 TTL、HttpOnly + Secure Cookie、各 org JWT secret 仍獨立隔離
App 影響	**完全無影響**（App WebView / React Native 不經過此流程）

進階選項（選配）：登入時同步預登入所有公會

登入成功後，後端同時為其他 4 個 org 生成 xorg token（TTL 300 秒），前端並行靜默 fetch 各 org 的 cross-login endpoint，使用者登入後即立刻在全部有帳號的公會建立 session。此方案需各 org 設定 CORS 與 `SameSite=None; Secure` ，有輕微安全折衷，建議 MVP 後視需求評估。

與整合計畫現有方案的差異對比

方案	各 org JWT secret	費用	需 redirect	App 影響
推薦：Redis Token Exchange	✅ 各自獨立	**$0**	一次（~100ms）	無
共用 JWT Signing Key	❌ 洩漏影響全體	$0	仍需 redirect	無
標準 OAuth2/OIDC	✅ 各自獨立	+新服務費用	標準流程	需改版

工時估算

工作項目	傳統工時	AI 協作工時
CrossOrgController（/crossorg/redirect + /auth/crossorg，khcpa PoC）	3 天	0.5 天
Redis xorg key 整合（確認不衝突、TTL 設計）	0.5 天	0.25 天
5 個前端改連結（AI 批次掃描 getOtherCpa() 相關呼叫）	3–4 天	0.75 天
AI 批次套用其餘 4 org 後端	1–2 天	0 天（AI 執行）
整合驗收（SSO 正常 / 無帳號跳登入頁 / token 一次性 / App 不受影響）	2 天	0.5 天
合計	**9–11 天**	**~2 天**

11.3 兩項功能合計

功能	傳統工時	AI 協作工時	節省比	額外月費
跨組織密碼同步	5–6 天	~1 天	~83%	$0
跨組織 SSO 自動登入	9–11 天	~2 天	~82%	$0
合計	**14–17 天**	**~3 天**	**~82%**	**$0**

> **前提條件：** 兩項功能均須在整合計畫完成後（JWT + Redis active session store 上線）方可實作。整合未完成前，現有 VM + HttpSession 架構無法以同等成本及架構達成相同效果。